

DAY 7 INTERNSHIP TECHNICAL REPORT

Algorithm Analysis, Complexities, and Big-O Notation

TRAINEE:	Benatic Rithan B	DEPARTMENT:	CSE
MENTOR:	Rathees G	DATE:	16 June 2026
INSTITUTION:	KIT, Coimbatore	STATUS:	Completed Successfully

1. EXECUTIVE SUMMARY

This technical report documents the algorithmic performance analysis conducted on Day 7 of the Python Backend Development Internship. Building directly upon the in-memory Inventory Management Tool engineered on Day 6, this analysis shifts the design focus from data structure selection to asymptotic runtime and auxiliary memory evaluation.

The primary objective is to mathematically audit existing code implementations using Big-O notation. By isolating execution steps and tracking memory overhead, we establish the baseline computational efficiency required to scale backend operations safely without introducing processing bottlenecks.

2. THEORETICAL FRAMEWORK & ANALYSIS RULES

To evaluate performance objectively, algorithmic behaviors are mapped against a theoretical efficiency hierarchy based on input size (n). Runtime calculations ignore hardware-dependent metrics (such as CPU clock speed in seconds) and instead count primitive operations. The structural analysis adheres to three core optimization laws:

- **Law of Dominant Terms:** Less significant terms are dropped to evaluate performance at a massive scale. For example, an algebraic runtime complexity of $O(n^2 + n)$ simplifies strictly to a quadratic complexity of $O(n^2)$.
- **Drop the Constants:** Scaling multipliers are discarded. A runtime of $O(2n)$ simplifies directly to linear growth $O(n)$.
- **Worst-Case Design Principle:** Algorithms are evaluated by their upper asymptotic bound, guaranteeing performance limits even under unfavorable data arrangements.

NOTATION	CLASSIFICATION	ASYMPTOTIC BEHAVIOR PROFILE	BACKEND IMPACT PROFILE
$O(1)$	Constant	Execution resources remain completely flat regardless of data scale.	Ideal benchmark performance.
$O(\log n)$	Logarithmic	Input data size is cleaved in half at each sequential operational step.	Highly efficient for deeply nested indexes.
$O(n)$	Linear	Processing costs grow in a strict 1:1 direct proportion to input volume.	Standard for simple sequential sweeps.
$O(n \log n)$	Linearithmic	Combines linear parsing with logarithmic slicing.	Standard runtime for highly optimized sorting algorithms.
$O(n^2)$	Quadratic	Resource demands square alongside input size; highly dangerous at scale.	Creates critical performance bottlenecks.

3. ALGORITHMIC COMPLEXITY AUDITS

Problem 1: Two Sum (Optimal Hash Map Strategy)

```
def twoSum(nums, target):
    seen = {} # Maps value to its index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

- **Time Complexity: $O(n)$**

Mechanistic Explanation: The algorithm leverages a single sequential pass over the array of size n . For every iterated element, calculating the `complement` takes $O(1)$ constant time. Checking keys inside the `seen` hash map (dictionary lookup) and inserting new keys both execute in an average time of $O(1)$. Because these constant-time actions sit inside a loop running n times, the overall growth scales linearly.

- **Space Complexity: $O(n)$**

Mechanistic Explanation: To achieve linear speed, the system trades memory resources. In the worst-case scenario where the target pairs sit at the absolute end of the collection or do not exist, the algorithm is forced to register every single element into the dictionary. The `seen` dictionary expands dynamically up to n entries, resulting in linear auxiliary space allocation.

Problem 2: Reverse String (Two-Pointer In-Place Strategy)

```
def reverseString(s):
    left = 0
    right = len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left] # Swap
        left += 1
        right -= 1
```

- **Time Complexity: $O(n)$**
Mechanistic Explanation: The architecture sets up opposing bounds-trackers (`left` and `right`) that systematically march toward the center index of the collection. The loop processes exactly $n/2$ iterations, where n represents the total character length of string array `s`. Applying the constant reduction rule, the fractional multiplier is dropped: $\frac{1}{2} \cdot O(n) \rightarrow O(n)$. The computational runtime scales in a clean, linear fashion.
- **Space Complexity: $O(1)$**
Mechanistic Explanation: This algorithm operates entirely "in-place," mutating the underlying reference buffer directly without making structural copies. Aside from initializing two low-overhead scalar pointer variables (`left` and `right`), the auxiliary memory usage remains perfectly flat regardless of whether the array holds ten characters or ten million characters.

4. ARCHITECTURAL TRADE-OFF ANALYSIS MATRIX

The systems analyzed showcase a fundamental engineering paradigm: trading memory to buy execution speed, or optimizing memory at the cost of processing sweeps.

ARCHITECTURAL COMPLEXITY COMPASS																							
															+ SPACE								
EFFICIENCY			^						O(1)		[Reverse String]			- Time: O(n)			- Space: O(1)						
- Strategy: In-place Swapping															O(n)		[Two Sum]			- Time: O(n)			-
Space: O(n)												- Strategy: Auxiliary Memory Cache											
															+-----> O(n) O(n^2) TIME								
EFFICIENCY		+-----+																					

- **Two Sum Architecture:** Prioritizes runtime optimization over memory containment. It avoids a naive nested-loop configuration ($O(n^2)$ time with $O(1)$ space) by introducing an auxiliary hash table cache. This lowers the execution threshold to a predictable linear $O(n)$ footprint, making it highly suitable for speed-critical lookup routes.
- **Reverse String Architecture:** Maximizes both time and space parameters simultaneously. By manipulating elements directly at their assigned memory address offsets, it eliminates system memory allocations entirely while cleanly completing the inversion in a single pass.

5. LEARNING OUTCOMES

- **Asymptotic Efficiency Assessment:** Mastered the capability to deduce runtime and memory scaling properties purely via static code inspection, shifting reliance away from unreliable, hardware-dependent execution timers.
- **Resource Trade-Off Literacy:** Developed a deep understanding of when to spend memory overhead (e.g., initializing hash tables) to compress quadratic execution loops into sub-linear or linear operational bounds.
- **Pointer Mutation Design:** Learned to apply structural traversal pointers (`left / right`) to manipulate collections in-place, preventing memory stack bloat and maintaining highly optimized $O(1)$ auxiliary space bounds.